

AI Interview Coach — End-to-End Flow (What & Why)

This document is for **demo and presentation**. It explains the full path a user takes through the system, **what component is used at each step**, and **why it exists**.

It does not replace code. Use it while walking the live app or architecture diagram.

1. One-sentence system story

A candidate signs in, provides a resume and job description, passes safety checks, gets a personalized voice interview run by cooperating agents, receives guarded feedback, and can download prep materials and revisit past sessions.

2. High-level end-to-end flow

```
User (Gradio UI)
  | login / upload or select Resume + JD
  ▼
Input Guardrails
  | reject unsafe / invalid documents
  ▼
Interview Manager (orchestrator)
  | owns state, order, routing
  ▼
Analysis Agent
  | matches resume ↔ JD, builds tailored questions
  ▼
Interview Agent ↔ Candidate answers (STT / TTS)
  | 6-question loop
  ▼
Evaluation Agent
  | full-session coaching feedback
  ▼
Output Guardrails
  | safety / bias check on feedback
  ▼
UI Output + Downloads + History
  score/feedback · PDF/DOCX · saved conversation
```

3. Step-by-step: What was used and why

Step 0 — Product shell (UI + auth)

What	Where	Why
Gradio (<code>app.py</code> , <code>ui/</code>)	Interactive web UI	Fast way to ship a demoable product (uploads, mic, audio, buttons) without building a full frontend
Auth module (<code>auth/</code>)	Email/password register & login	Ties documents and past interviews to a person; needed for “my library / my history”
bcrypt hashing	Stored in <code>auth/users.json</code>	Passwords must not be stored in plain text
Header user chip	Shows signed-in identity	Makes the session feel like a real product, not a raw notebook

Demo line:

“Users don’t just chat with a model — they have an account so their practice materials persist.”

Step 1 — Document intake (HITL)

What	Where	Why
File upload (PDF/DOCX/TXT)	Gradio File components	Real candidates already have resume/JD files
<code>pypdf</code> / <code>python-docx</code>	<code>app.read_uploaded_text()</code>	Extract text so agents can reason over content
Document library (<code>library/documents.py</code>)	Save / reuse / delete per user	Avoid re-uploading every session; supports repeat practice
Auto-save uploads	Checkbox in UI	Convenience without forcing a separate “save” click

Demo line:

“Human-in-the-loop starts here: the candidate chooses which resume and which job to practice against.”

Step 2 — Input Guardrails (safety gate before agents)

What	Where	Why
InputGuardrails	<code>agents/guardrails.py</code>	Diagram Step 2: block bad inputs <i>before</i> analysis burns tokens
Rule checks	Empty/short docs, injection-like patterns	Fast, cheap, deterministic first line of defense
LLM safety check	Same module	Catches nuanced unsafe content rules alone might miss

What	Where	Why
Wired in	<code>InterviewManagerAgent.start()</code>	Orchestrator refuses to start if gate fails

Demo line:

"We don't analyze everything blindly. Input guardrails protect the pipeline."

If blocked, user sees: status like 🛑 `Input guardrail blocked start: ...`

Step 3 — Interview Manager (orchestration)

What	Where	Why
InterviewManagerAgent	<code>agents/interview_manager_agent.py</code>	Single coordinator: state, turn order, agent calls, guardrail hooks
InterviewSession dataclass	Same file	Keeps resume/JD, questions, answers, audio paths in one place
Not one giant prompt	Design choice	Separation of concerns: analysis ≠ interviewing ≠ evaluation

Demo line:

"The LLM writes language; the manager owns workflow control — that's the agentic pattern."

Step 4 — Analysis Agent (personalization)

What	Where	Why
JobAnalysisAgent	<code>agents/job_analysis_agent.py</code>	Compare resume vs JD: skills, gaps, strengths
Structured JSON output	Analysis response	Reliable handoff of tailored questions into the session
Question bank helpers	<code>questions/</code>	Fixed opening/closing + fallbacks if generation fails

Demo line:

"Questions are grounded in *this* resume and *this* job — not random interview trivia."

Step 5 — Interview loop (Interview Agent + speech)

What	Where	Why
InterviewAgent	<code>agents/interview_agent.py</code>	Serves next question from the prepared list
6-question flow	Manager + question bank	Predictable demo: opening → 4 tailored → candidate asks a question
OpenAI TTS	<code>text_to_speech()</code> in <code>app.py</code>	Makes practice feel like a real interview
OpenAI STT (transcribe)	<code>transcribe_audio()</code>	Candidates answer by voice; system stores text for evaluation
Answer persistence in session	Manager <code>evaluate_answer()</code>	Full transcript needed for final review

Demo line:

"Voice in, voice out — closer to real interview pressure than typing into a chat box."

Step 6 — Evaluation Agent (full-session judgment)

What	Where	Why
EvaluationAgent	<code>agents/evaluation_agent.py</code>	Scores after <i>all</i> answers (holistic coaching, not fragmented mid-turn noise)
Uses analysis context	Prompt includes job analysis	Feedback stays role-relevant

Demo line:

"We evaluate the whole conversation so feedback reflects consistency across answers."

Step 7 — Output Guardrails (safety gate before UI/report)

What	Where	Why
OutputGuardrails	<code>agents/guardrails.py</code>	Diagram output gate: check feedback for unsafe/biased content
Rewrite-if-needed	Same module	Prefer safe coaching text over hard failure when possible
Wired in	After <code>evaluate_interview()</code> inside Manager	User never sees unvalidated final feedback

Demo line:

"Even model-written coaching goes through an output safety check before display and download."

Step 8 — Results, downloads, and history

What	Where	Why
Feedback panel	UI results section	Immediate coaching value after the interview
Reports service	<code>reports/service.py</code>	Build downloadable artifacts
PDF / DOCX exporters	<code>reports/exporters.py</code> (fpdf2, python-docx)	Candidates leave with files they can keep
Study pack agent	<code>reports/study_pack_agent.py</code>	Expected questions + strong answers + concept explanations
Conversation store	<code>library/conversations.py</code>	Persist Q&A + audio under <code>user_data/</code>
History UI	Library card in UI	Re-read past interviews; replay question/answer audio
Naming	<code>conversation_title()</code>	Meaningful labels like <code>JD - 2026-10-09 19:13</code>

Demo line:

"Practice isn't disposable — users can download a study pack and reopen past conversations."

4. Map to folders (for "show me the code" moments)

```

app.py           → Gradio wiring, STT/TTS, start/answer handlers
agents/
  interview_manager_agent.py → orchestration + session state
  job_analysis_agent.py     → personalization
  interview_agent.py        → question delivery
  evaluation_agent.py       → final coaching
  guardrails.py             → input + output safety gates
  base_agent.py             → shared OpenAI call helper
auth/               → register/login, hashed passwords, header UI
library/            → resumes, JDs, conversation history
reports/            → feedback + study pack downloads
ui/                 → layout + CSS
questions/          → fixed/fallback question bank
user_data/          → per-user persisted files (local)

```

5. "Why multi-agent?" (short answer for judges)

Single mega-prompt	This design
Hard to debug	Each agent has one job

Single mega-prompt	This design
Unclear control flow	Manager owns deterministic steps
Easy to mix safety + coaching	Guardrails are explicit gates
Weak product features	Library/history/downloads sit cleanly around agents

Line to say:
"The model generates language; the agents and manager enforce structure, state, and safety."

6. Live demo checklist (aligned to this flow)

- 1. Register/login → show user chip
- 2. Upload resume + JD (or pick saved) → mention library
- 3. Start → say "input guardrails passed" when status shows it
- 4. Answer 1–2 questions with mic → transcribe → save
- 5. Finish (or open past interview) → show feedback
- 6. Download PDF/DOCX study pack
- 7. Open Past interviews → play an audio clip

Backup if live API is slow: open a saved conversation and narrate Steps 5–8 from history.

7. Technologies summary (quick slide companion)

Technology	Role in this project
Python	Core application and agent orchestration
Gradio	Demo UI (files, mic, audio, downloads)
OpenAI API	Analysis, interview language, STT, TTS, evaluation, study pack
Multi-agent pattern	Specialized roles + manager orchestration
Input/Output guardrails	Safety before analysis and before feedback delivery
Local <code>user_data/</code>	Per-user resumes, JDs, conversations, audio
PDF/DOCX tooling	Read uploads and export coaching artifacts

8. Closing proof points

If someone asks "What did you actually build?", answer with these four proofs:

- 1. **Personalized questions** from resume + JD (Analysis Agent)
- 2. **Orchestrated interview loop** with session state (Interview Manager)
- 3. **Safety gates** on input and output (Guardrails)
- 4. **Product persistence** — library, history, downloadable prep pack

That is the end-to-end value of the project.